

CS & IT ENGINEERING

Programming in C

Functions and Storage Classes

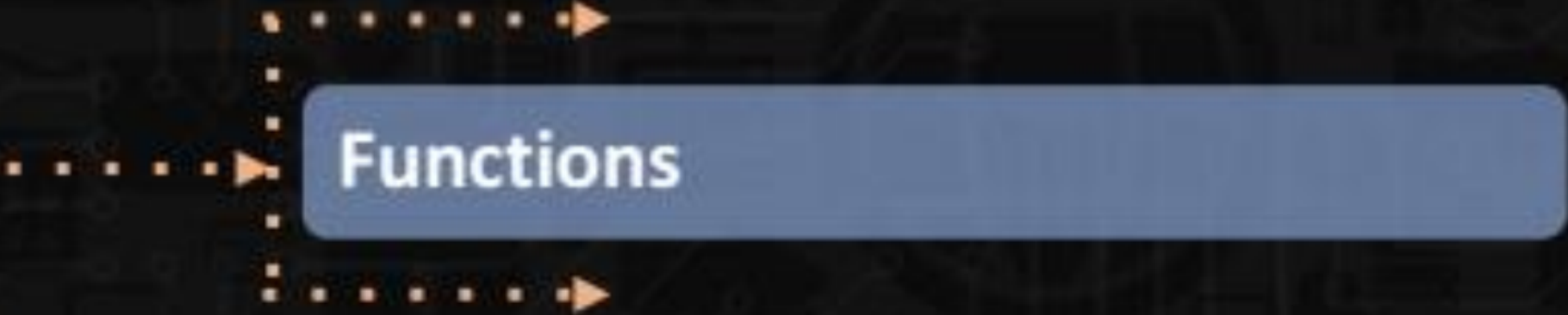
Lec- 01



By- Pankaj Sharma sir



TOPICS TO BE
COVERED

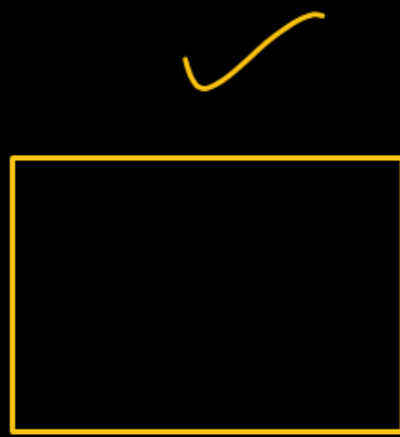


Functions

Function



modular programming



code
is not
correct

```
#include <stdio.h>
```

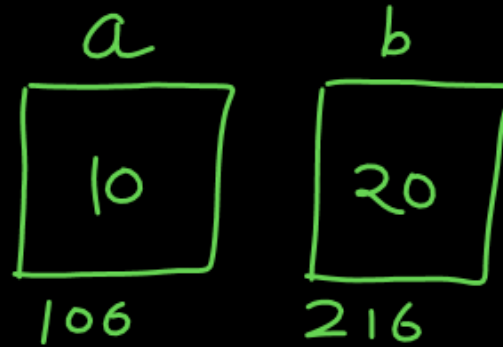
```
void main() {
```

```
    int a = 10, b = 20, result;
```

```
    result = satishsir(10, 20a, b);
```

```
    printf("%d", result);
```

```
}
```



```
satishsir(int x, int y)
```

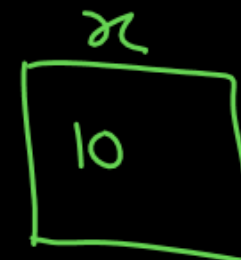
```
{
```

```
    int temp;
```

```
    temp = x * y;
```

```
    return temp;
```

```
}
```




```
#include<stdio.h>

void main(){

printf("%d",a);

}
```

a ?
what is a ?
without
declaration
using

```
#include<stdio.h>

void main(){

printf("Pankaj");

}
```

forward
declaration

?

```
#include <stdio.h>
```

```
int Add(int, int);
```

forward declaration

```
void main(){
```

```
    int a=10, b=20, result;
```

```
    result = Add(a, b); // using Add  
                // Calling Add
```

```
    printf("%d", result);
```

```
}
```

```
int Add(int x, int y)
```

return type

Code/
body/
definition

```
{  
    int sum;
```

```
    sum = x + y;
```

```
    return sum;
```

```
}
```

```
#include <stdio.h>
```

```
int Add(int, int);
```

forward declaration

Return type of Add func.
is int.

Happy

```
int Add(int x, int y)
```

```
{  
    int sum;
```

```
    sum = x + y;
```

```
    return sum;
```

```
}
```

```
void main(){
```

```
    int a = 10, b = 20, result;
```

```
    result = Add(a, b);
```

```
    printf("%d", result);
```

```
}
```

by default
the return
type is
int


```
#include <stdio.h>
```

```
void main() {
```

```
    int x, y = 10;
```

```
    x = fun(y);
```

```
    printf("%d", x);
```

```
}
```

Compiler will assume that the return type of fun() is int

Angry Error

```
double fun(int a)
{
    double temp = 3.2;
    return temp * a;
}
```

```
#include <stdio.h>
```

```
void main(){
```

```
    int a=10, b=20, result;
```

```
    result = Add(a, b);
```

```
    printf("%d", result);
```

```
}
```

Compiler will assume that the return type for Add func is int

Happy, dancing

```
int Add(int x, int y)
{
    return x+y;
}
```

```
#include<stdio.h>
void main(){

}
```

Compile ✓
Execute ✓

```
#include<stdio.h>
int Add(int,int);
```

Compile ✓
Execute ✗

forward declaration

```
#include <stdio.h>
void main(){
    printf("Hello");
}
```

Pankaj.c

Pre
processing

```

extern int printf(
    const char*, ...)
void main(){
    printf("Hello");
}
```

Call Pankaj.i

forward declaration

Assembly
Code

Pankaj.s

Assembler

Object

Pankaj.o

scanf.o

printf.o

Linker

Pankaj.out
(runnable)

```
#include <stdio.h>
```

```
int Add(int, int);
```

```
void main() {
```

```
1. int a = 10, b = 20, result;
```

```
2. result = Add(a, b);
```

```
3. printf("%d", result);
```

```
4. }
```

30

gnu compiler collection
(gcc)

called

```
int Add(int x, int y)
```

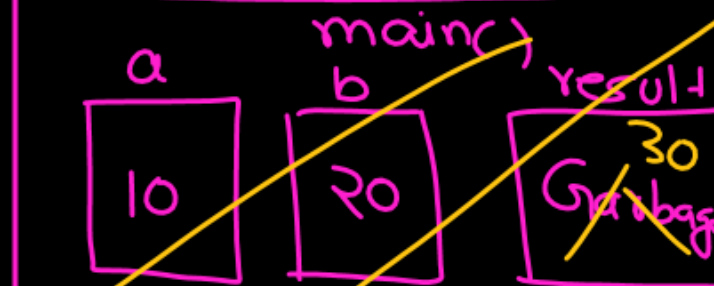
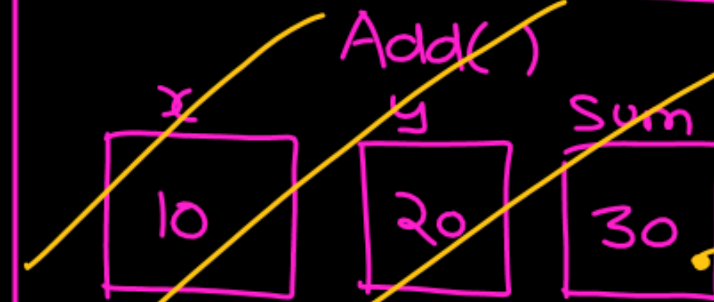
```
{
```

```
    int sum;
```

```
    sum = x + y;
```

```
    return sum;
```

```
}
```



main()

Activation record

```
#include <stdio.h>
```

```
void main(){
```

```
    int a=10, b=20, result;
```

```
    result = Add10 20(a, b);
```

actual values

```
    printf("/d", result);  
}
```

actual parameters,
actual arguments

formal argument

```
int Add(int x, int y)
```

```
{
```

```
    int sum;
```

```
    sum = x + y;
```

```
    return sum;
```

```
}
```



```
#include <stdio.h>
```

```
void f(){
```

```
1. printf("1");
```

```
2. }
```

```
void g(){
```

```
1. printf("2")
```

```
2. f();
```

```
3. printf("3");
```

```
4. }
```

```
void main(){
```

```
1. g();
```

```
2. g();
```

```
3. }
```

213213

main

1 ✓
2 ✓
3 ✓

```
void f(){
    1 printf("1");
```

2 3

```
void g(){
```

```
    1 printf("2");
```

```
    2 f();
```

```
    3 printf("2");
```

```
    4 }
```

```
void h(){
```

```
    1 printf("3");
```

```
    2 g();
```

```
    3 printf("3");
```

```
    4 }
```

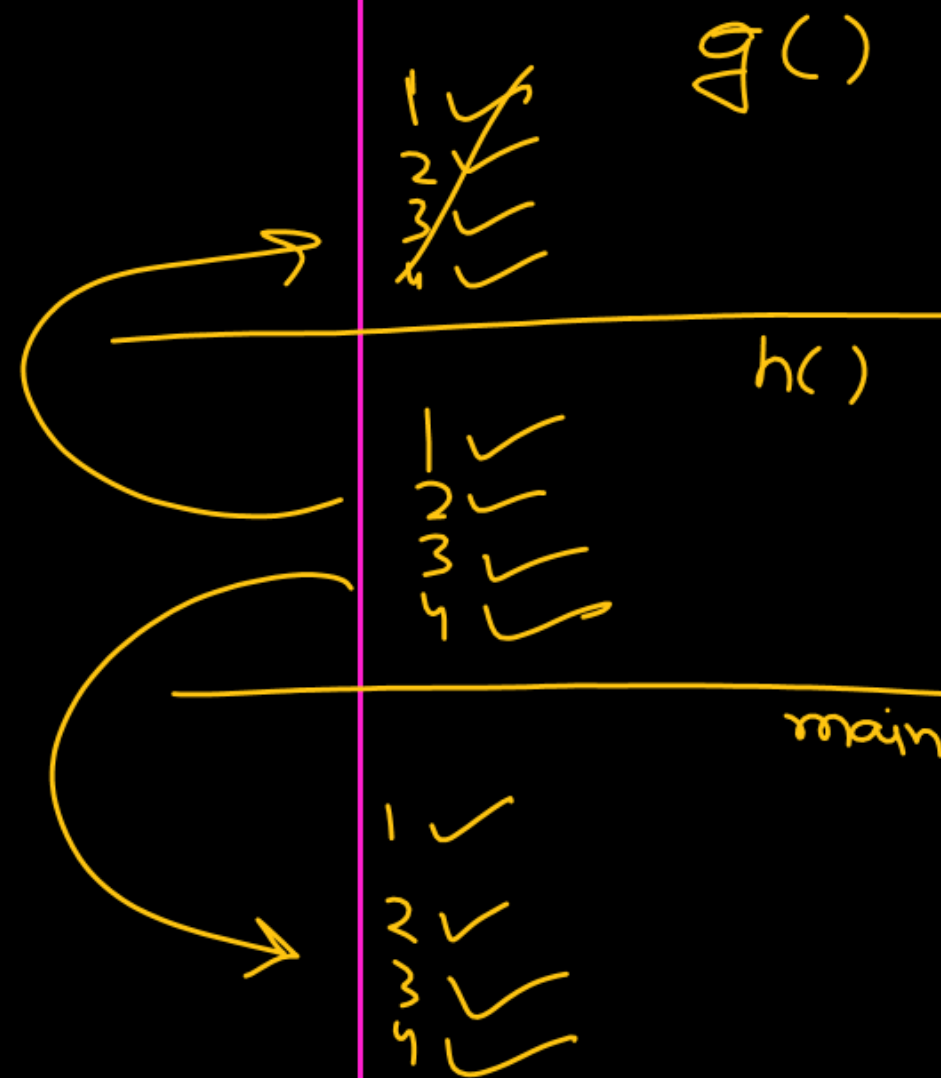
```
void main(){
```

```
    1 h();
```

```
    2 g();
```

```
    3 h();
```

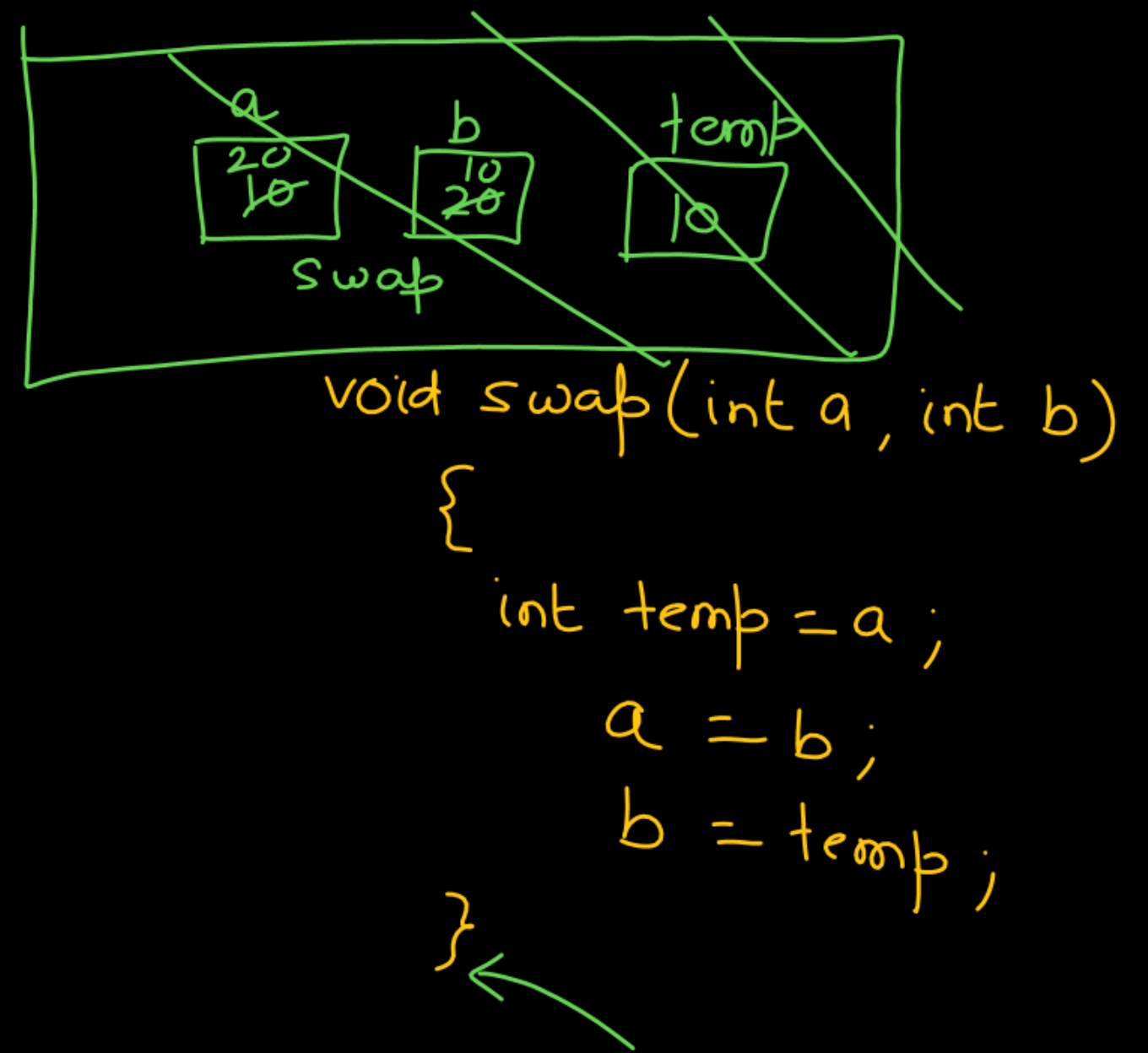
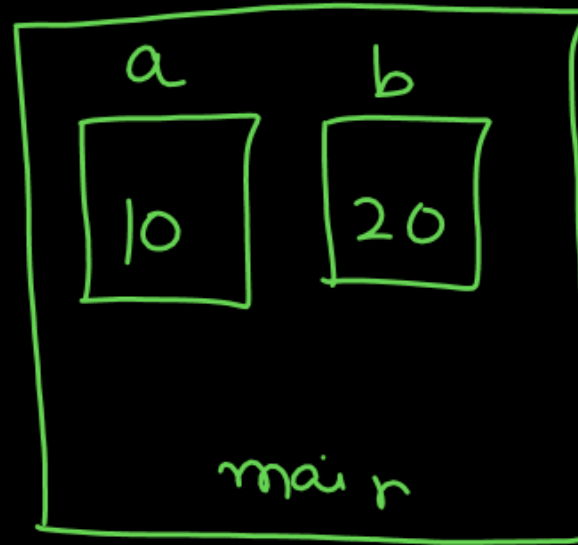
32123212
32123



```

#include <stdio.h>
void swap(int, int);
void main() {
    int a = 10, b = 20;
    swap(10a, 20b);
    printf("/d /d", a, b);
}

```



8:30 PM

